AI-Powered Escape Room: design and implementation report

Submitted By: Sathyamurthi Karthikeyan

Supervised By: Iftikhar Ahmed

1. Objective and experience

The project implements a conversational escape room in Python, using Streamlit for interaction and Google Gemini as the language model. The player wakes up in the Midnight Library and explores five objects: a desk, bookshelf, painting, locked drawer and exit door. A short introduction explains the setting; the sidebar gives command examples, model settings and a restart button. Conversation remains visible throughout the session. Version 2 also displays authoritative inventory and puzzle progress.

Three connected puzzles give the game a clear solution. First, the desk instructs the player to count moons, suns and stars. The bookshelf supplies three, one and four respectively, giving the drawer combination 314. Second, the player must collect the brass key from the unlocked drawer and use it on the painting. Third, the painting reveals numbers assigned to seasons. Ordering them according to the desk's winter–spring–summer–autumn instruction produces the exit code 7249. The door also checks that the drawer and painting sequence is complete. Guessing the exit code at the start cannot bypass those prerequisites.

2. Version 1: Conversational Baseline

Version 1 sends the game configuration, puzzle solutions and full conversation to Gemini. A system instruction asks it to narrate concisely, remember prior actions and enforce prerequisites. The model controls the story's progression and emits an escape marker when it believes the player has won. Streamlit stores messages in session state and recognizes that marker to finish the game.

3. Version 2: Explicit rules and state

Version 2 separates language interpretation, state transitions and presentation. A data class holds collected items, examined objects, solved puzzles, drawer status, painting status, completion and turn count. Puzzle constants and all prerequisites remain in Python. The action interpreter receives the current player message and an allowed action vocabulary, but no secret combinations or mutable state. It returns a small JSON object describing a requested action, arbitrary state fields are discarded.

Python validates every action before changing the state. It prevents collecting a key through the locked drawer, using an uncollected key, unlocking the painting with an unsupported item, opening non-existent objects and completing the exit early. Code attempts must match an explicit numeric token in the user's message, preventing the interpreter from manufacturing a solution. Repeating successful actions does not duplicate inventory or solved-puzzle entries. The escaped flag is set only by the validated final door action.

Free-form generated narration could still contradict a correct engine. To close this gap the Version 2 uses constrained narration: Gemini selects one of two approved descriptions built from the validated outcome. Only a valid numeric selection is accepted, malformed or arbitrary output falls back to the canonical description. This deliberately sacrifices narrative variety for reliable player-visible behavior. The language model still interprets requests and selects the presentation, while Python determines success. Generated prose never becomes a source of state or rules.

4. Implementation challenges and trade-offs

The main design challenge was choosing a useful boundary for an unreliable component. Merely adding structured state would not prevent an unrestricted narrator from saying that the door opened. Constrained output closes that particular failure path and keeps undiscovered secrets out of the interpreter's prompt. Nevertheless, the interpreter can misread natural language. Version 2 guarantees validation of the proposed action, not perfect understanding of user intent. It is not a general security sandbox or a formal proof against every possible injection.

Provider failures were encountered during user testing: invalid keys, unavailable model IDs (404), rejected requests (400) and temporary service errors (503). The app now discovers model IDs, excludes incompatible Antigravity agents, and gives safe error categories. Version 1 retries a missing model once with another discovered Flash text candidate; generation retries temporary server failures once after one second. Failed turns preserve conversation. Version 2 displays a persistent warning when falling back to Python. Credentials are never written to the repository, and raw provider errors are not displayed. Session data remains temporary.

User testing also exposed overly strict commands: "examining the desk", "bookshelf" and "count moons" were rejected. The parser now accepts these phrases and provides guidance for season references without unlocking anything. It remains simpler than Gemini and supports one action per turn. Future work could add richer approved narration, confirmations for ambiguous requests, a larger room configuration format and saved games. Changes that depend on a provider still require available project access and quota; listing a model does not prove free access or generation success.

5. Evaluation and deliverables

The automated suite contains 36 test methods covering a complete escape, invalid combinations, missing inventory, false history, administrator claims, nonexistent objects, repeated actions and premature exit attempts. Model-boundary tests verify that invented codes and arbitrary narration cannot corrupt the validated result. Streamlit interaction tests cover restart, completion, model selection, recovery and fallback warnings. All 36 tests pass without real API requests.

Seven controlled fault-injection comparisons demonstrate the architectural difference. Deliberately faulty model fixtures simulate missing inventory, wrong codes, false history, administrator impersonation, secret disclosure, invented objects and premature escape. Version 1 passes the injected narration through; Version 2 rejects invalid state transitions and unapproved narration. All seven Version 2 integrity checks pass. The recorded responses and state snapshots are provided in the comparison-results files.

These comparisons demonstrate software behavior under simulated faults, not Gemini's measured live failure rate. User-shared gameplay shows live narration and provider errors; a complete live adversarial comparison remains unrecorded. The optional evaluation command can capture real responses using the user's Gemini project. The delivered project includes both application versions, SVG and JPG architecture diagrams, comparative cases, test evidence and this three-page report.